

AI Assisted Azure Data Factory Data Engineering Exercise

An overview of building a mock financial reporting solution using Azure Data Factory and Power BI

Author: Al Franco

Date: July 2026

Category: Portfolio Project | **Status:** Completed

Contents

1. Project Overview
2. Architecture Overview
3. Azure Data Model & Schema
4. Power BI Interface
5. Key Learnings and Next Steps
6. Appendix: Resources & References

1. Project Overview

This project demonstrates an AI-assisted data engineering exercise, including the design and implementation of a financial reporting dashboard. It utilizes a medallion architecture design pattern and is built with Azure Data Factory pipelines as the ETL component. Power BI is employed to create the dashboard with a direct connection to Azure SQL database “Gold layer” views. The primary goal of this project is to learn more about how a seasoned data management professional can leverage AI models to accelerate the design and build of a common BI reporting cloud-based use case. In addition, I am leveraging this project as a step in a continued effort to quickly upskill on cloud platforms and solutions.

From a business context, this project delivers a reporting solution for two core financial statements — an Income Statement and a Balance Sheet. As a starting point, I utilize dimension and fact data files generated by AI to support a fictional Bank’s P&L and Balance Sheet dashboards. The AI-generated data is fictitious and does not represent any financial institution’s data. The fictitious data is used primarily to accelerate the design and build of the solution and is not of high quality. In the future, data quality will be improved if available synthetic data is incorporated.

Preparing for this effort:

In order to prepare for the Azure component of this exercise, I took several Microsoft “Azure Fundamentals” and “Well Architected Framework” courses.

AI assisted design and build:

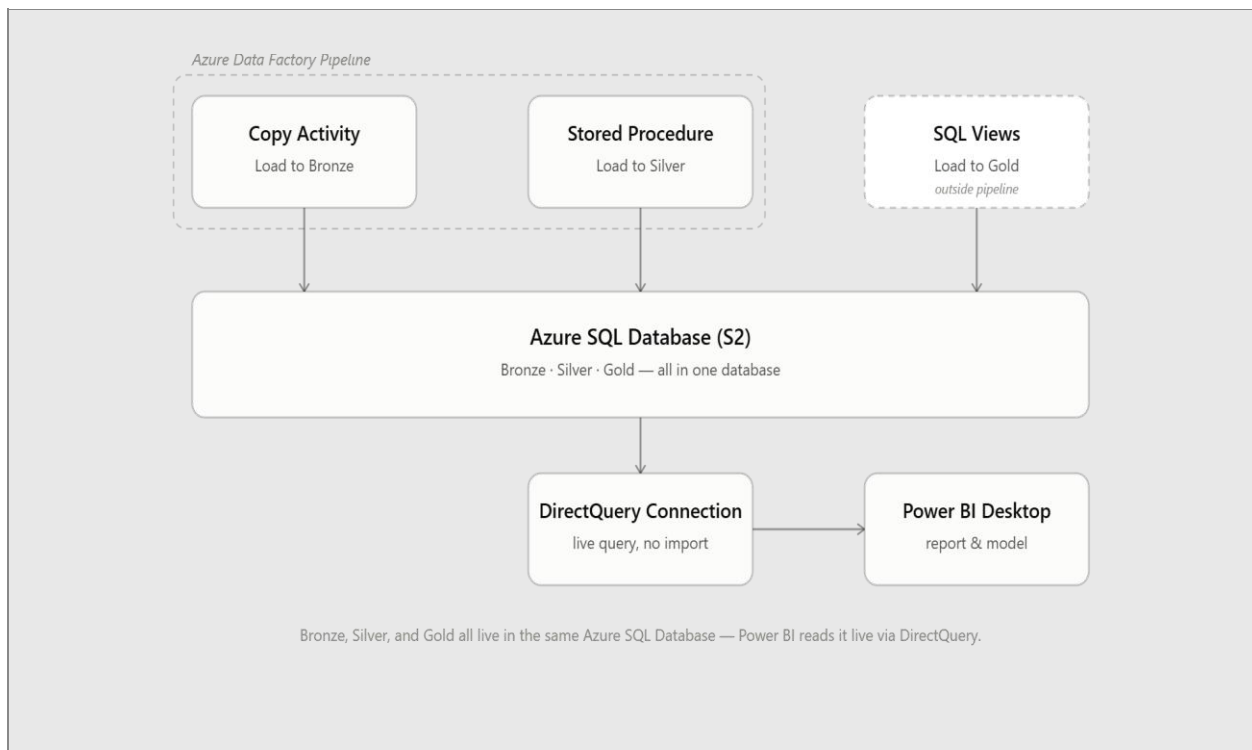
I uploaded datasets to AI and requested that it incorporate a medallion architecture with views for the Gold layer. I also requested for AI to use Azure Data Factory for the ETL engine and Power BI as the reporting layer. The choices made by AI looked reasonable: Pipelines to load to Bronze tables, Stored Procedures to map from Bronze to Silver tables, then views for the Gold layer. AI also provided DDL and stored procedures for the Azure SQL DB. It provided some DAX measures and recommendations for the P&L and Balance Sheet page layouts. I was able to relatively quickly upskill on Azure Data Factory and build a functioning solution. I worked through iterations of the solution design, set up Azure, configured connections, utilized additional tools, corrected issues in the generated output, worked through compatibility issues, and added more tables/views/pipelines/DAX measures. *(see “A1 AI Data Model Rationale (generated by data model)” in the Appendix)*

2. Architecture Overview

2.1 Solution Architecture

The solution follows a straightforward, cloud-native architecture. Financial transaction data loaded by Azure Data Factory and is stored and managed in an **Azure SQL Database** (S2 service tier), which acts as the authoritative data store. Power BI Desktop connects to this database via a **DirectQuery** connection, meaning no data is imported or cached locally — every visual refresh issues a live T-SQL query to the Azure SQL engine.

2.2 Architecture Diagram



2.3 Technology Stack

Component	Technology
Cloud Database	Azure SQL Database (S2 tier)
ETL Tool	Azure Data Factory (Pipelines)
Reporting Tool	Microsoft Power BI Desktop
Connection Mode	DirectQuery
Query Language	T-SQL (Views & Stored Procedures)
DAX Engine	Power BI Analysis Services
SQL Interface to Azure SQL	SQL Server Management Studio Visual Studio Code – Jupyter Notebook

3. Azure Data Model & Schemas

3.1 Schema Overview (Bronze and Silver)

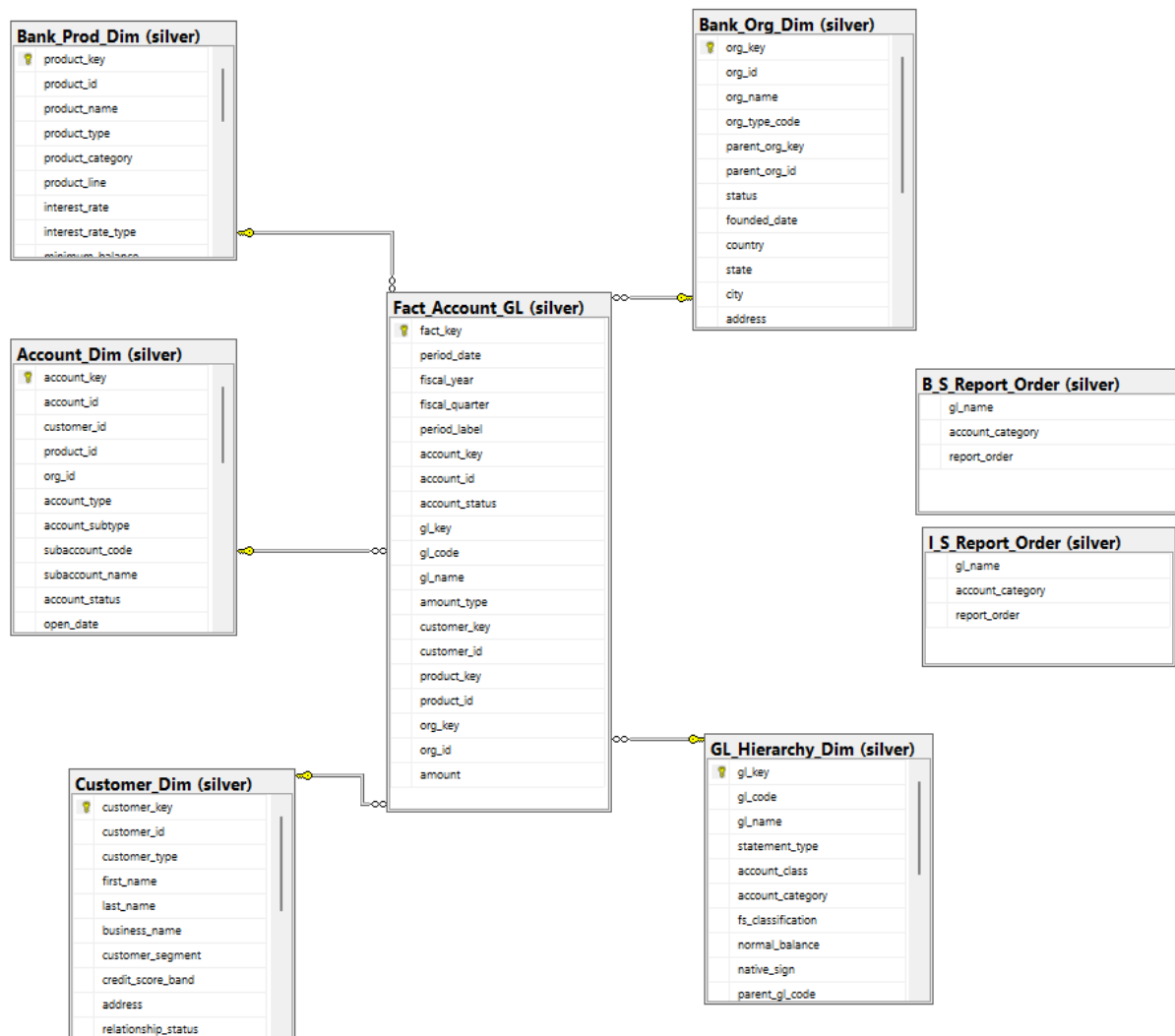
The database schema follows a classic star-schema pattern optimized for financial reporting workloads. At the center of the Silver schema sits a primary fact table: `silver.Fact_Account_GL` which captures the balances that drive the reports. Surrounding the fact table are dimension tables that provide descriptive context to the rows on the fact table that can be used for reporting: `silver.GL_Hierarchy_DIM` defines chart of accounts / financial statement structure that drive the Balance Sheet and Income Statement Rollups; `silver.Bank_Org_Dim` defines organizational units; `silver.Bank_Prod_Dim` provides the attributes associated with the product catalog (e.g Premier Checking, Basic Savings); `silver.Customer_Dim` represents the bank's customers (individual and business) and `silver.Account_Dim` is the account master (sub-ledger) and is comprised of one row per real customer-owned bank account (checking, savings, loan, etc.). All dimension tables use surrogate integer keys for join performance, with natural business keys retained as indexed alternate keys.

stg_Balance_Sheet_Order (bronze) gl_name account_category report_order _load_ts	stg_Income_Statement_Order (bronze) gl_name account_category report_order _load_ts		
stg_Customer_Dim_SCD (bronze) customer_key customer_id customer_type first_name last_name business_name customer_segment credit_score_band address relationship_status kyc_status aml_risk_rating primary_org_id eff_start_date eff_end_date is_current version_number created_ts	stg_Bank_Org_Dim_SCD (bronze) org_key org_id org_name org_type_code parent_org_key parent_org_id status founded_date country state city address phone email eff_start_date eff_end_date is_current version_number created_ts	stg_Bank_Prod_Dim_SCD (bronze) product_key product_id product_name product_type product_category product_line interest_rate interest_rate_type minimum_balance monthly_fee fee_waiver_min_balance account_term_months product_tier overdraft_protection_flag digital_banking_flag rewards_program_flag fdic_insured_flag product_status regulatory_classification	stg_Account_Dim_SCD (bronze) account_key account_id customer_id product_id org_id account_type account_subtype subaccount_code subaccount_name account_status open_date interest_rate_type collateral_type bs_account_code bs_account_name bs_account_category bs_account_subcategory bs_rollup_code bs_rollup_name
stg_Fact_Account_GL (bronze) fact_key period_date fiscal_year fiscal_quarter period_label account_key account_id account_status gl_key gl_code gl_name amount_type customer_key customer_id product_key	stg_GL_Hierarchy_Dim_SCD (bronze) gl_key gl_code gl_name statement_type account_class account_category fs_classification normal_balance native_sign parent_gl_code hierarchy_level is_leaf is_contra is_derived		

3.1.1 Azure SQL DB Bronze Layer

Notes

- 1) **bronze.stg_Balance_Sheet_order & bronze.stg_Income_Statement_Order** were built to support Power BI reporting rollups implemented in DAX measures.
- 2) Bronze tables contained rows in SCD table that the AI model elected to filter as it moved to the Silver layer.
- 3) See Section “A2) Azure SQL DB Bronze Layer DDL” in the Appendix for the details.

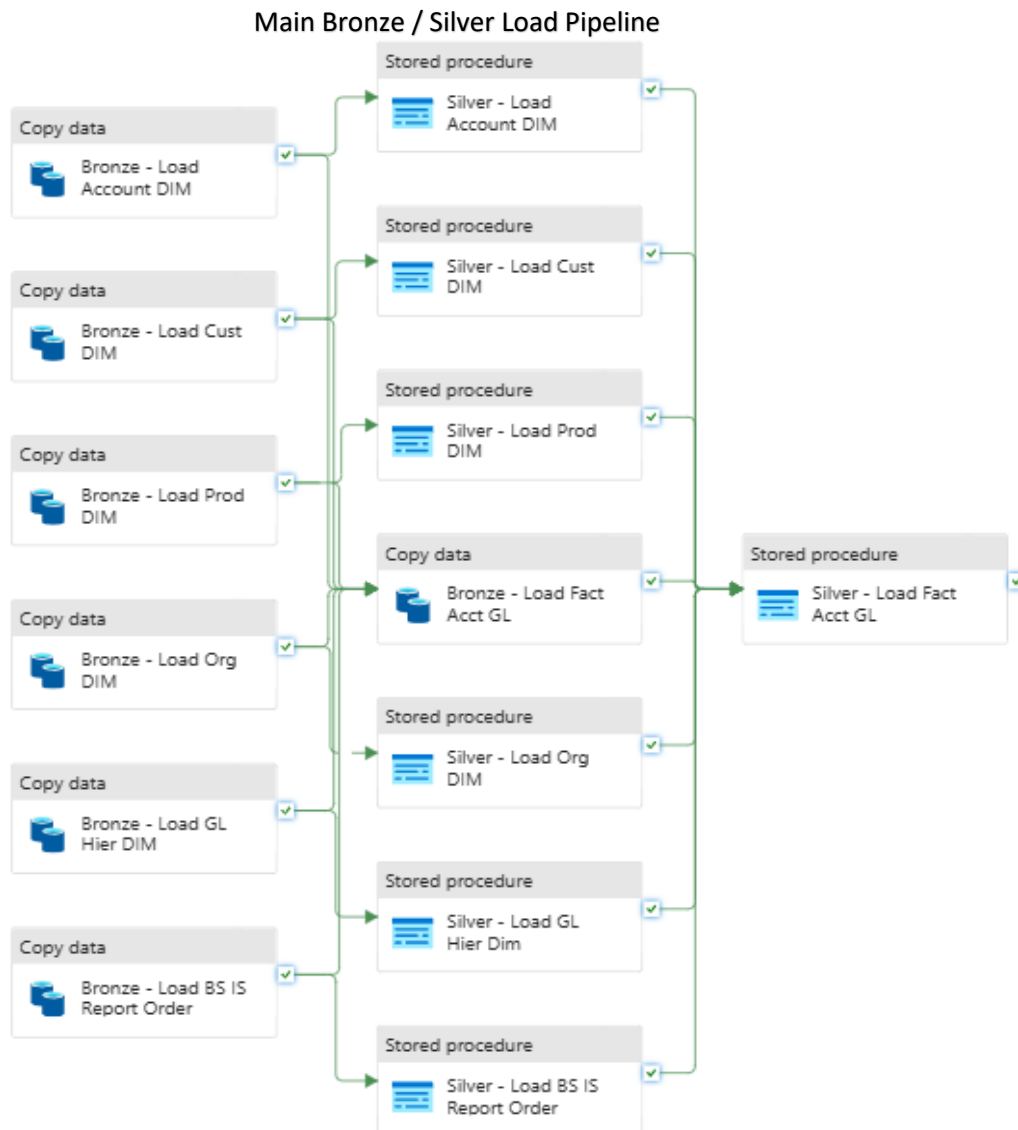


Azure SQL DB Silver Layer

Notes

- 1) silver.B_S_Report_Order & silver.I_S_Report_Order were built to support Power BI reporting rollups implemented in DAX measures.
- 2) The above schema was mostly designed by the AI tool and that normalization design decisions are outlined in the Appendix under “AI Data Model Rationale (generated by data model)”
- 3) See Section “A3) Azure SQL DB Silver Layer DDL” in the Appendix for the details

3.2 Bronze and Silver Pipelines



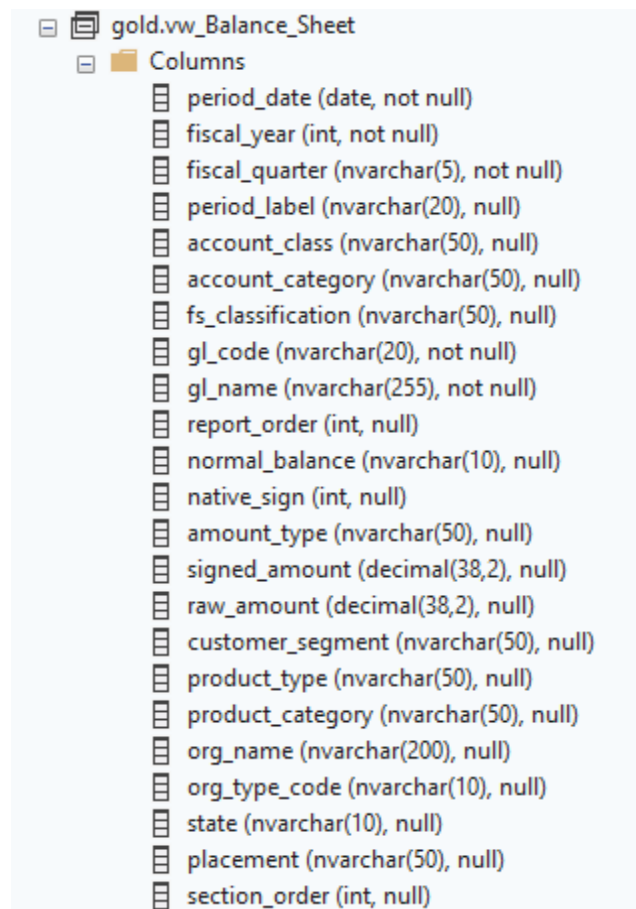
Notes

- 1) The Main Bronze/Silver Load Pipeline was built to load the Bronze and Silver data tables.
- 2) Additional pipelines were built in the same fashion to load the Balance Sheet and Income Statement report order data that was needed by DAX
- 3) See Section "A4) Azure SQL DB Silver Layer Stored Procedures" in the Appendix for the details

3.3 Gold Layer SQL Views

Rather than exposing raw fact and dimension tables directly to Power BI, the solution introduces a dedicated SQL view layer. These views pre-aggregate GL transaction data, apply account filtering logic, and compute derived fields (such as running balances for balance sheet accounts) entirely within the Azure SQL engine. This approach keeps complex transformation logic server-side and out of Power Query — reducing the M code footprint in Power BI to near zero — and ensures that all business rules are centrally maintained and version-controlled in SQL scripts. For DirectQuery specifically, this is critical: each view is designed so that Power BI's generated T-SQL folds cleanly against it, enabling query plan optimization and index utilization.

Balance Sheet View:



gold.vw_Balance_Sheet
Columns
period_date (date, not null)
fiscal_year (int, not null)
fiscal_quarter (nvarchar(5), not null)
period_label (nvarchar(20), null)
account_class (nvarchar(50), null)
account_category (nvarchar(50), null)
fs_classification (nvarchar(50), null)
gl_code (nvarchar(20), not null)
gl_name (nvarchar(255), not null)
report_order (int, null)
normal_balance (nvarchar(10), null)
native_sign (int, null)
amount_type (nvarchar(50), null)
signed_amount (decimal(38,2), null)
raw_amount (decimal(38,2), null)
customer_segment (nvarchar(50), null)
product_type (nvarchar(50), null)
product_category (nvarchar(50), null)
org_name (nvarchar(200), null)
org_type_code (nvarchar(10), null)
state (nvarchar(10), null)
placement (nvarchar(50), null)
section_order (int, null)

Income Statement View

gold.vw_Income_Statement
Columns
period_date (date, not null)
fiscal_year (int, not null)
fiscal_quarter (nvarchar(5), not null)
period_label (nvarchar(20), null)
account_class (nvarchar(50), null)
account_category (nvarchar(50), null)
fs_classification (nvarchar(50), null)
gl_code (nvarchar(20), not null)
gl_name (nvarchar(255), not null)
report_order (int, null)
normal_balance (nvarchar(10), null)
native_sign (int, null)
is_contra (bit, null)
amount_type (nvarchar(50), null)
signed_amount (decimal(38,2), null)
raw_amount (decimal(38,2), null)
customer_segment (nvarchar(50), null)
credit_score_band (nvarchar(20), null)
product_type (nvarchar(50), null)
product_category (nvarchar(50), null)
product_line (nvarchar(50), null)
org_name (nvarchar(200), null)
org_type_code (nvarchar(10), null)
state (nvarchar(10), null)
city (nvarchar(100), null)
placement (nvarchar(50), null)
section_order (int, null)

Income Statement Report Order View for DAX

gold.vw_B_S_Report_Order
Columns
gl_name (nvarchar(255), null)
account_category (nvarchar(50), null)
report_order (int, null)

Balance Sheet Report Order View for DAX

gold.vw_I_S_Report_Order
Columns
gl_name (nvarchar(255), null)
account_category (nvarchar(50), null)
report_order (int, null)

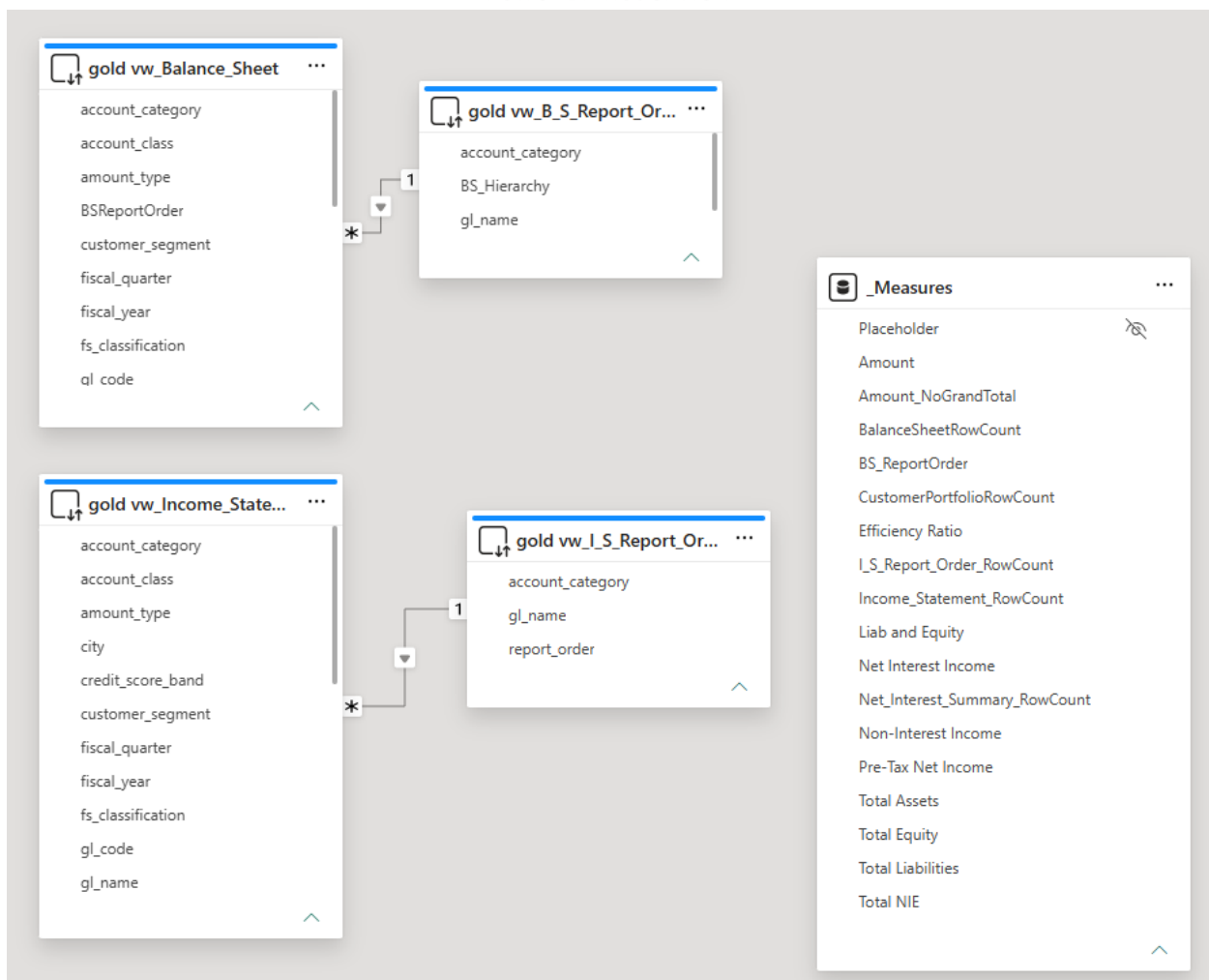
Note – see Section A5) Azure SQL DB Gold Layer Views in the appendix for details

4. Power BI Interface

4.1 Integration with Azure SQL Database

Power BI Desktop is connected directly to Azure SQL DB Gold layer views. The gold vw_Balance_Sheet and gold vw_Income_Statements drive their respective visualizations. Due to some limitations with DAX measures, the AI model recommended that I create specialized views to enable a DAX measure-based report order within the Power BI visualizations. The remaining Measures were suggested by the AI model. Some (not all) were utilized in Visualizations.

Power BI Model View



4.2 Balance Sheet Page

The AI model provided the outline of a balance sheet page with Quarter, Org Unit and Customer Segment Slicers. The report is driven by “gold vw_Balance_Sheet” and “gold vw_B_S_Report_Order” Azure SQL DB views via DirectQuery. I also had to build some DAX measures to support the sort order and Balance Sheet category rollups (1. Assets, 2.Liabilities, 3.Equity)

Quarter		Balance Sheet (Fictitious)					
☐ Q1 2025 ☐ Q2 2025 ☐ Q3 2025 ☐ Q4 2025							
Org Unit							
☐ Ashford Branch ☐ Chestnut Ridge Branch ☐ Copperstone Branch ☐ Creekside Branch ☐ Downtown Core Branch ☐ Eastview Branch ☐ Elmwood Branch ☐ Fairview Branch							
Cust Segment							
☐ (Blank) ☐ Commercial ☐ Corporate ☐ Premium ☐ Private Banking ☐ Retail ☐ SMB							
		Balance Sheet	Q1 2025	Q2 2025	Q3 2025	Q4 2025	Total
		1. Assets	135,948,922.14	138,952,296.76	170,617,331.93	172,939,223.70	618,457,774.53
		Cash and Due from Banks	40,730,618.41	9,633,124.30	59,708,261.06	44,588,756.45	154,660,760.22
		Available-for-Sale Securities (AFS)	14,092,050.14	74,298,694.28	71,649,016.78	61,038,293.70	221,078,054.90
		Held-to-Maturity Securities (HTM)	62,801,702.58	33,416,359.85	14,559,223.96	45,250,861.98	156,028,148.37
		Residential Mortgage Loans — 1-4 Family	5,561,745.54	4,860,282.51	4,673,439.22	5,082,222.31	20,177,689.58
		Home Equity Lines of Credit	614,011.14	768,442.31	769,800.84	567,813.37	2,720,067.66
		Consumer Installment Loans	468,143.37	382,216.29	427,028.65	517,087.59	1,794,475.90
		Credit Card Receivables	182,583.80	156,748.74	185,494.30	137,187.57	662,014.41
		Commercial Lines of Credit	1,144,152.12	1,016,602.65	1,747,968.40	1,794,338.37	5,703,061.54
		Less: Allowance for Credit Losses (ACL)	-84,024.85	-69,545.49	-66,725.40	-35,711.17	-256,006.91
		Accrued Interest Receivable	782,899.14	1,441,566.81	244,589.68	1,912,969.32	4,382,024.95
		Deferred Tax Asset (DTA)	811,175.73	771,928.31	735,636.16	906,899.11	3,225,639.31
		Goodwill and Intangible Assets	2,547,562.14	4,374,447.52	3,184,530.51	1,599,143.45	11,705,683.62
		Premises and Equipment, Net	3,744,965.44	5,551,097.69	9,273,572.20	7,991,529.47	26,561,164.80
		Other Real Estate Owned (OREO)	1,766,866.39	1,509,671.14	1,989,616.95	1,263,533.02	6,529,687.50
		Other Assets	784,471.05	840,659.85	1,535,878.62	324,299.16	3,485,308.68
		2. Liabilities	90,742,016.64	49,775,586.89	83,093,192.39	70,406,063.88	294,016,859.80
		3. Equity	45,206,905.50	89,176,709.87	87,524,139.54	102,533,159.82	324,440,914.73
		Common Stock and Additional Paid-In Capital	10,948,186.28	67,911,627.29	30,306,137.28	25,467,580.48	134,633,531.33
		Retained Earnings	23,190,853.33	14,871,216.80	52,159,558.81	71,855,467.42	162,077,096.36
		Accumulated Other Comprehensive Income (AOCI)	2,876,594.03	2,855,490.73	2,386,360.07	3,467,776.04	11,586,220.87
		Treasury Stock	8,191,271.86	3,538,375.05	2,672,083.38	1,742,335.88	16,144,066.17
		Total	271,897,844.28	277,904,593.52	341,234,663.86	345,878,447.40	1,236,915,549.06

DAX Measures

“gold vw_B_S_Report_Order” =

BS_Hierarchy =

SWITCH(

TRUE(),

'gold vw_B_S_Report_Order'[account_category] = "Asset", "1. Assets",

'gold vw_B_S_Report_Order'[account_category] = "Liability", "2. Liabilities",

'gold vw_B_S_Report_Order'[account_category] = "Equity", "3. Equity",

"4. Other"

)

“gold vw_Balance_Sheet” =

BSReportOrder = RELATED('gold vw_B_S_Report_Order'[report_order])

4.3 P&L Matrix Page

The AI model provided the outline of a Profit and Loss page with Quarter, Org Unit and Customer Segment Slicers. The report is driven by “gold vw_Income_Statement” and “gold vw_I_S_Report_Order” Azure SQL DB views via DirectQuery. I also had to build a DAX measure to support the sort order. Also, the AI model prescribed a few DAX measures for the page.

Quarter

Q1 2025

Q2 2025

Q3 2025

Q4 2025

Org Unit

Ashford Branch

Chestnut Ridge Branch

Copperstone Branch

Creekside Branch

Cust Segment

(Blank)

Commercial

Corporate

Premium

Income Statement (Fictitious)

Income Statement	Q1 2025	Q2 2025	Q3 2025	Q4 2025	Total
Revenue	\$689,397.33	\$458,888.35	\$656,829.79	\$877,313.93	\$2,682,429.4
Interest Income — Residential Mortgage Loans	\$82,046.2	\$65,870.76	\$67,695.69	\$68,234.62	\$283,847.27
Interest Income — Home Equity Lines of Credit	\$12,395.94	\$16,019.91	\$15,176.76	\$11,820.69	\$55,413.3
Interest Income — Consumer Installment Loans	\$19,792.2	\$16,286.58	\$17,691.98	\$21,818.45	\$75,589.21
Interest Income — Credit Card Receivables	\$9,056.07	\$7,390.39	\$8,864.04	\$6,329.26	\$31,639.76
Interest Income — Commercial Lines of Credit	\$32,940.11	\$28,203.6	\$46,663.99	\$46,727.93	\$154,535.63
Interest Income — AFS Securities	\$302,673.31	\$112,565	\$392,542.08	\$430,676.97	\$1,238,457.36
Interest Income — HTM Securities	\$193,540.36	\$175,017	\$82,568.75	\$253,459.88	\$704,585.99
Service Charges on Deposit Accounts	\$6,150.08	\$5,131.26	\$3,577.3	\$5,364.9	\$20,223.54
Card Fees and Interchange Income	\$1,983.26	\$2,199.19	\$2,976.93	\$1,683.33	\$8,842.71
Commercial Service Charges	\$26,400.06	\$27,792.17	\$16,667.03	\$28,799.87	\$99,659.13
Total	\$689,397.33	\$458,888.35	\$656,829.79	\$877,313.93	\$2,682,429.4

	Q1 2025	Q2 2025	Q3 2025	Q4 2025	Total
Expense	(\$4,844,557.52)	(\$4,856,243.23)	(\$5,728,313.86)	(\$3,346,801.62)	(\$18,775,916.23)
Interest Expense — Demand Deposits (DDA)	(\$224.59)	(\$293.81)	(\$313.21)	(\$235.35)	(\$1,066.96)
Interest Expense — Savings Deposits	(\$721.51)	(\$735.18)	(\$786.84)	(\$672.46)	(\$2,915.99)
Interest Expense — High-Yield Savings Deposits	(\$3,143.35)	(\$5,417.39)	(\$2,528.23)	(\$3,603.14)	(\$14,692.11)
Interest Expense — Money Market Deposits	(\$3,896.25)	(\$3,197.17)	(\$2,435.01)	(\$3,237.1)	(\$12,765.53)
Interest Expense — Time Deposits (CDs)	(\$4,597.2)	(\$4,141.26)	(\$4,005.75)	(\$6,352.97)	(\$19,097.18)
Interest Expense — Business Demand Deposits	(\$3,050.21)	(\$3,938.45)	(\$3,076.39)	(\$3,090.29)	(\$13,155.34)
Interest Expense — Federal Funds Purchased	(\$126,121.89)	(\$122,682.77)	(\$149,000.9)	(\$101,665.33)	(\$499,470.89)
Interest Expense — FHLB Advances	(\$307,533.73)	(\$264,101.18)	(\$341,611.45)	(\$256,708.36)	(\$1,169,954.72)
Interest Expense — Subordinated Debt	(\$175,399.45)	(\$51,144.33)	(\$186,538.24)	(\$74,758.66)	(\$487,840.68)
Provision — Real Estate Loans	(\$17,023.11)	(\$15,617.82)	(\$17,065.85)	(\$19,720.75)	(\$69,427.53)
Provision — Consumer Loans	(\$4,805.5)	(\$4,356.69)	(\$5,220.26)	(\$4,542.41)	(\$18,924.86)
Provision — Commercial Loans	(\$6,313.5)	(\$4,567.71)	(\$8,483.36)	(\$8,909.02)	(\$28,273.59)
Salaries and Employee Benefits	(\$1,922,628.92)	(\$2,729,865.74)	(\$2,524,196.57)	(\$708,527.8)	(\$7,885,219.03)
Total	(\$4,844,557.52)	(\$4,856,243.23)	(\$5,728,313.86)	(\$3,346,801.62)	(\$18,775,916.23)

\$323.11K
Net Interest Income

\$138.36K
Non-Interest Income

(\$13.96M)
Total NIE

\$14.42M
Pre-Tax Net Income

-3024.69%
Efficiency Ratio

<

DAX Measures

“_Measures” =

Net Interest Income =

```
CALCULATE(
    SUM('gold vw_Income_Statement'[signed_amount]),
    'gold vw_Income_Statement'[account_class] IN { "Interest Income", "Interest Expense" }
)
```

Non-Interest Income =

```
CALCULATE(
    SUM('gold vw_Income_Statement'[signed_amount]),
    'gold vw_Income_Statement'[account_class] = "Non-Interest Income"
)
```

“_Measures” =

Total NIE =

```
CALCULATE(  
    SUM('gold vw_Income_Statement'[signed_amount]),  
    'gold vw_Income_Statement'[account_class] = "Non-Interest Expense"  
)
```

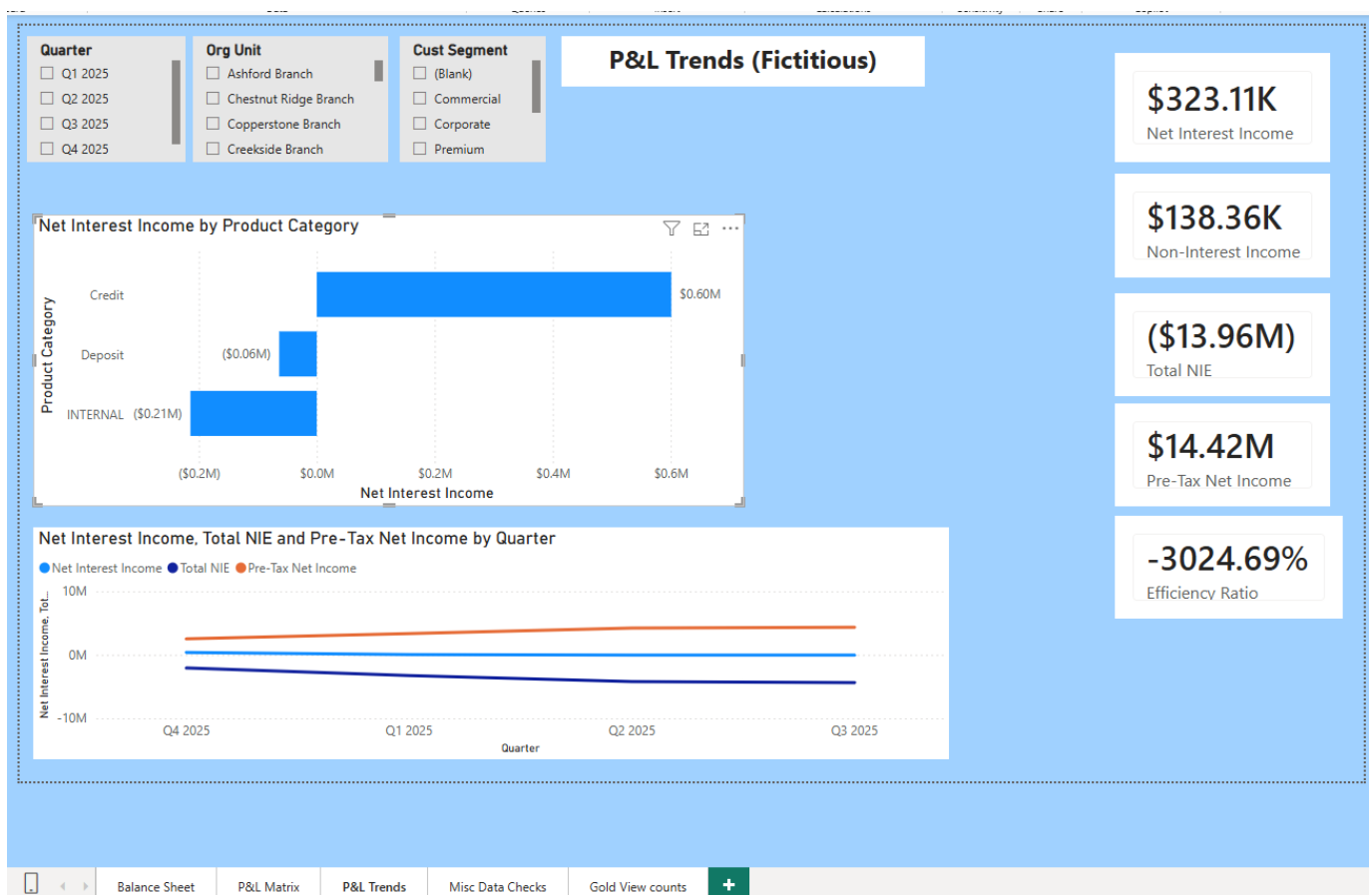
Pre-Tax Net Income = [Net Interest Income] + [Non-Interest Income] - [Total NIE]

Efficiency Ratio =

DIVIDE([Total NIE], [Net Interest Income] + [Non-Interest Income])

4.4 P&L Trends Page

The AI model provided the outline of a Profit and Loss Trends page with Quarter, Org Unit and Customer Segment Slicers. The report is driven by the “gold vw_Income_Statement” Azure SQL DB views via DirectQuery and the DAX measures outlined in the previous section.



5. Key Learnings

5.1 Key Learnings

This project served as learning exercise. I focused on utilizing AI models to outline a design and approach for a standard BI use case that is familiar to me. As an experienced professional in the data and analytics space, I have designed solutions and continually upskill on various toolsets and platforms without the assistance of Agentic AI. I embraced the opportunity to leverage AI to dive into the Azure Data Factory platform. Below are some of my key takeaways from the exercise:

- **Learning the fundamentals of Azure Data Factory is the first step:** Use available training resources to understand the Azure platform and the how to employ it's compute, storage, networking, data & analytics products and security to addressing your use case with a solid architecture. AI model output is not helpful if you are not grounded in the platform.
- **A foundation in data engineering, solution architecture and data governance principles is a pre-requisite:** To take advantage of the full design/build acceleration offered by Agentic AI for this use case, you must be grounded in:
 - Data modelling, medallion architecture, ETL pipelines, network connectivity, database builds, data governance, data quality, solution architecture, and data security (for example).
 - A requirements-based understanding what approaches and components are acceptable in a solution design/architecture to meet user requirements.

With the above foundation, Agentic AI can serve as a powerful partner.

- **Bottom line is that I was able to go from scratch to a working cloud-base BI Dashboard using Azure Data Factory pipelines in less than 2 weeks.**
 - The data model and generated Table DDL, Stored Procedures view definitions were pretty complete and useable by Azure and Power BI. I liked the data model that was generated with some denormalization in the Account_Dim that I had not seen before. I

also like AI's decision to utilize stored procedures as the ETL tool for Silver layer transformations. Most generated DAX measures worked. The Power BI report layouts were pretty easy to build.

- In order to achieve the required result, I had to tweak the models and approaches as well as debug some of the content generated by AI. In addition, a couple of the original DAX measures did not work with the DirectQuery views so I had to address the issue with additional views in the Gold layer (an approach suggested by AI)
- **It is easier to continually make progress as AI can be utilized to help you work through issues and offer different approaches:** It was harder to “stay stuck” on a topic or task.
- **Product offerings are always changing:** When I embarked on this effort to accelerate my learning of the Microsoft Azure Data Factory (ADF), it was clear that Fabric Data Factory is the “next generation” of ADF. Online resources steered me to Fabric Data Factory, but I stayed with ADF.

Appendix: Resources & References

A1) AI Data Model Rationale (generated by data model):

Business purpose of each table

Fact_Account_GL — the transactional fact table. Each row is one GL posting for one bank account in one period: a balance, an interest expense, a fee, etc. (amount_type tells you which). This is what actually gets summed to build the Balance Sheet and P&L in Power BI — everything else exists to filter, label, and roll it up.

Account_Dim_SCD — the account master (sub-ledger). One row per real customer-owned bank account (checking, savings, loan, etc.), describing the account itself: type/subtype, open date, rate type, collateral — plus which BS/IS statement line(s) it *normally* reports to. It's a slowly-changing dimension (eff_start_date/eff_end_date/is_current), so history is preserved when an account's attributes change.

GL_Hierarchy_Dim_SCD — the chart of accounts / financial-statement structure. One row per GL code (e.g., BS-1100 Cash and Due from Banks, IS-5100 Interest Expense), with parent_gl_code, hierarchy_level, statement_type (BS vs IS), normal_balance, report_order. This is what drives the actual layout and subtotal rollups of the Balance Sheet and P&L.

Bank_Prod_Dim_SCD — the product catalog (Premier Checking, Basic Savings, etc.), with rate, fee, and eligibility attributes — used to slice reporting by product line/tier.

Bank_Org_Dim_SCD — the organizational hierarchy (HQ → Region → branch), used to slice/report by business unit.

Customer_Dim_SCD — the customer master (individual/business, segment, credit band, KYC/AML status) — used to slice by customer attributes and support relationship-level reporting.

Account_Dim_SCD's account vs. GL_Hierarchy_Dim_SCD's account

These represent two different real-world things that happen to both get called "account":

- Account_Dim_SCD.account_id (e.g., ACC0001) is a customer's bank account — a sub-ledger entity. It's specific to one customer, one product, one branch. Thousands of these exist, and many share the same GL classification.
 - GL_Hierarchy_Dim_SCD.gl_code (e.g., BS-2110, IS-5100) is a general ledger line item — a slot in the chart of accounts / financial statement, not tied to any customer. Every checking account's interest expense posts to the *same* IS-5100 code, no matter which of the 1,000 customers owns the account.
-

The link between

them: Account_Dim_SCD carries bs_account_code/primary_is_code/secondary_is_code as the account's *default* GL destinations, while Fact_Account_GL.gl_key records where each specific transaction actually posted (balance → BS code, interest expense → primary IS code, fees → secondary IS code, etc.). So the account dimension answers "whose account is this," and the GL hierarchy dimension answers "where does it land on the financial statements."

Why customer_id/product_id/org_id appear in both Fact_Account_GL and Account_Dim_SCD

This is intentional denormalization, not an error. In a strict snowflake schema you'd only need account_key on the fact table, and Power BI would have to hop Fact → Account_Dim → Customer_Dim to filter by customer. Instead, this design pushes customer_key/customer_id, product_key/product_id, and org_key/org_id directly onto Fact_Account_GL as their own foreign keys pointing straight at Customer_Dim_SCD, Bank_Prod_Dim_SCD, and Bank_Org_Dim_SCD.

Why do it this way:

1. True star schema, not snowflake — every dimension relates directly to the fact table in one hop. Power BI's filter propagation and DAX are simplest and fastest when the model is a clean star; multi-hop chains through Account_Dim_SCD would work but add join depth and risk ambiguous/inactive relationship paths.
 2. Independent slicing — you can filter the fact table by customer or org or product *without* going through the account dimension at all, which matters because those are legitimately separate business dimensions of a GL posting, not just attributes of the account.
 3. SCD stability — Account_Dim_SCD is itself slowly changing (an account's product or org could change over time via eff_start_date/eff_end_date). Carrying the natural ids on the fact row lets each posting stay tied to the customer/product/org that was true *at the time of that transaction*, independent of how the account dimension's current row looks.
-

The coexistence of the surrogate key (_key) and natural id (_id) on the fact table follows the same logic each time: the _key is the join/relationship column to the dimension, the _id is the human-readable business key kept for traceability and debugging.

A2) Azure SQL DB Bronze Layer DDL

```
CREATE SCHEMA bronze;
GO

CREATE TABLE bronze.stg_Fact_Account_GL (
    fact_key INT,
    period_date NVARCHAR(50),
    fiscal_year NVARCHAR(10),
    fiscal_quarter NVARCHAR(10),
    period_label NVARCHAR(20),
    account_key INT,
    account_id NVARCHAR(20),
    account_status NVARCHAR(20),
    gl_key INT,
    gl_code NVARCHAR(20),
    gl_name NVARCHAR(255),
    amount_type NVARCHAR(50),
    customer_key INT,
    customer_id NVARCHAR(20),
    product_key INT,
    product_id NVARCHAR(20),
    org_key INT,
    org_id NVARCHAR(20),
    amount DECIMAL(18,2),
    _load_ts DATETIME2 DEFAULT GETUTCDATE()
);

CREATE TABLE bronze.stg_GL_Hierarchy_Dim_SCD (
    gl_key INT,
    gl_code NVARCHAR(20),
    gl_name NVARCHAR(255),
    statement_type NVARCHAR(10),
    account_class NVARCHAR(50),
    account_category NVARCHAR(50),
    fs_classification NVARCHAR(50),
    normal_balance NVARCHAR(10),
    native_sign INT,
    parent_gl_code NVARCHAR(20),
    hierarchy_level INT,
    is_leaf NVARCHAR(10),
    is_contra NVARCHAR(10),
    is_derived NVARCHAR(10),
    report_order INT,
    maps_to_product NVARCHAR(50),
    eff_start_date NVARCHAR(50),
    eff_end_date NVARCHAR(50),
    is_current NVARCHAR(10),
    _load_ts DATETIME2 DEFAULT GETUTCDATE()
);
```

```

CREATE TABLE bronze.stg_Account_Dim_SCD (
  account_key          INT,
  account_id           NVARCHAR(20),
  customer_id          NVARCHAR(20),
  product_id           NVARCHAR(20),
  org_id               NVARCHAR(20),
  account_type         NVARCHAR(50),
  account_subtype      NVARCHAR(50),
  subaccount_code      NVARCHAR(20),
  subaccount_name      NVARCHAR(100),
  account_status       NVARCHAR(20),
  open_date            NVARCHAR(50),
  interest_rate_type   NVARCHAR(20),
  collateral_type       NVARCHAR(50),
  bs_account_code       NVARCHAR(20),
  bs_account_name      NVARCHAR(255),
  bs_account_category  NVARCHAR(50),
  bs_account_subcategory NVARCHAR(100),
  bs_rollup_code        NVARCHAR(20),
  bs_rollup_name        NVARCHAR(100),
  acl_code             NVARCHAR(20),
  acl_name             NVARCHAR(100),
  primary_is_code       NVARCHAR(20),
  primary_is_name      NVARCHAR(255),
  primary_is_category  NVARCHAR(50),
  secondary_is_code     NVARCHAR(20),
  secondary_is_name     NVARCHAR(255),
  eff_start_date        NVARCHAR(50),
  eff_end_date          NVARCHAR(50),
  is_current            NVARCHAR(10),
  _load_ts             DATETIME2 DEFAULT GETUTCDATE()
);

```

```

CREATE TABLE bronze.stg_Bank_Prod_Dim_SCD (
  product_key          INT,
  product_id           NVARCHAR(20),
  product_name         NVARCHAR(100),
  product_type         NVARCHAR(50),
  product_category     NVARCHAR(50),
  product_line         NVARCHAR(50),
  interest_rate         DECIMAL(10,6),
  interest_rate_type   NVARCHAR(20),
  minimum_balance      DECIMAL(18,2),
  monthly_fee          DECIMAL(10,2),
  fee_waiver_min_balance DECIMAL(18,2),
  account_term_months  INT,
  product_tier         NVARCHAR(20),
  overdraft_protection_flag NVARCHAR(5),
  digital_banking_flag NVARCHAR(5),
  rewards_program_flag NVARCHAR(5),
  fdic_insured_flag    NVARCHAR(5),
  product_status       NVARCHAR(20),
  regulatory_classification NVARCHAR(50),
  source_system        NVARCHAR(50),
  eff_start_date        NVARCHAR(50),
  eff_end_date          NVARCHAR(50),
  is_current            NVARCHAR(10),
  created_ts           NVARCHAR(50),
  updated_ts           NVARCHAR(50),
  _load_ts             DATETIME2 DEFAULT GETUTCDATE()
);

```

```

CREATE TABLE bronze.stg_Bank_Org_Dim_SCD (
  org_key      INT,
  org_id       INT,
  org_name     NVARCHAR(200),
  org_type_code NVARCHAR(10),
  parent_org_key INT,
  parent_org_id INT,
  status       NVARCHAR(20),
  founded_date NVARCHAR(50),
  country      NVARCHAR(50),
  state        NVARCHAR(10),
  city         NVARCHAR(100),
  address      NVARCHAR(255),
  phone        NVARCHAR(30),
  email        NVARCHAR(100),
  eff_start_date NVARCHAR(50),
  eff_end_date  NVARCHAR(50),
  is_current    NVARCHAR(10),
  version_number INT,
  created_ts    NVARCHAR(50),
  updated_ts    NVARCHAR(50),
  _load_ts     DATETIME2 DEFAULT GETUTCDATE()
);

CREATE TABLE bronze.stg_Customer_Dim_SCD (
  customer_key      INT,
  customer_id       NVARCHAR(20),
  customer_type     NVARCHAR(20),
  first_name        NVARCHAR(100),
  last_name         NVARCHAR(100),
  business_name     NVARCHAR(200),
  customer_segment  NVARCHAR(50),
  credit_score_band NVARCHAR(20),
  address           NVARCHAR(255),
  relationship_status NVARCHAR(20),
  kyc_status        NVARCHAR(20),
  aml_risk_rating   NVARCHAR(20),
  primary_org_id    NVARCHAR(20),
  eff_start_date    NVARCHAR(50),
  eff_end_date      NVARCHAR(50),
  is_current        NVARCHAR(10),
  version_number    INT,
  created_ts        NVARCHAR(50),
  updated_ts        NVARCHAR(50),
  _load_ts         DATETIME2 DEFAULT GETUTCDATE()
);

CREATE TABLE bronze.stg_Income_Statement_Order (
  gl_name      NVARCHAR(255),
  account_category NVARCHAR(50),
  report_order INT,
  _load_ts     DATETIME2 DEFAULT GETUTCDATE()
);

CREATE TABLE bronze.stg_Balance_Sheet_Order (
  gl_name      NVARCHAR(255),
  account_category NVARCHAR(50),
  report_order INT,
  _load_ts     DATETIME2 DEFAULT GETUTCDATE()
);

```

A3) Azure SQL DB Silver Layer DDL

```
CREATE SCHEMA silver;
GO

CREATE TABLE silver.GL_Hierarchy_Dim (
    gl_key          INT          NOT NULL,
    gl_code         NVARCHAR(20) NOT NULL,
    gl_name         NVARCHAR(255) NOT NULL,
    statement_type  NVARCHAR(10) NOT NULL,
    account_class   NVARCHAR(50),
    account_category NVARCHAR(50),
    fs_classification NVARCHAR(50),
    normal_balance  NVARCHAR(10),
    native_sign     INT,
    parent_gl_code  NVARCHAR(20),
    hierarchy_level INT,
    is_leaf         BIT,
    is_contra       BIT,
    is_derived      BIT,
    report_order    INT,
    maps_to_product NVARCHAR(50),
    eff_start_date  DATE          NOT NULL,
    eff_end_date    DATE          NOT NULL,
    is_current      BIT          NOT NULL,
    CONSTRAINT PK_GL_Hierarchy PRIMARY KEY (gl_key)
);

CREATE TABLE silver.Account_Dim (
    account_key      INT          NOT NULL,
    account_id       NVARCHAR(20) NOT NULL,
    customer_id      NVARCHAR(20),
    product_id       NVARCHAR(20),
    org_id           NVARCHAR(20),
    account_type     NVARCHAR(50),
    account_subtype  NVARCHAR(50),
    subaccount_code  NVARCHAR(20),
    subaccount_name  NVARCHAR(100),
    account_status   NVARCHAR(20),
    open_date        DATE,
    interest_rate_type NVARCHAR(20),
    collateral_type  NVARCHAR(50),
    bs_account_code  NVARCHAR(20),
    bs_account_name  NVARCHAR(255),
    bs_account_category NVARCHAR(50),
    bs_account_subcategory NVARCHAR(100),
    bs_rollup_code   NVARCHAR(20),
    bs_rollup_name   NVARCHAR(100),
    acl_code         NVARCHAR(20),
    acl_name         NVARCHAR(100),
    primary_is_code  NVARCHAR(20),
    primary_is_name  NVARCHAR(255),
    primary_is_category NVARCHAR(50),
    secondary_is_code NVARCHAR(20),
    secondary_is_name NVARCHAR(255),
    eff_start_date   DATE          NOT NULL,
    eff_end_date     DATE          NOT NULL,
    is_current       BIT          NOT NULL,
    CONSTRAINT PK_Account_Dim PRIMARY KEY (account_key)
);
```

```

CREATE TABLE silver.Bank_Prod_Dim (
  product_key      INT      NOT NULL,
  product_id       NVARCHAR(20) NOT NULL,
  product_name     NVARCHAR(100),
  product_type     NVARCHAR(50),
  product_category NVARCHAR(50),
  product_line     NVARCHAR(50),
  interest_rate    DECIMAL(10,6),
  interest_rate_type NVARCHAR(20),
  minimum_balance  DECIMAL(18,2),
  monthly_fee      DECIMAL(10,2),
  fee_waiver_min_balance DECIMAL(18,2),
  account_term_months INT,
  product_tier     NVARCHAR(20),
  overdraft_protection_flag BIT,
  digital_banking_flag BIT,
  rewards_program_flag BIT,
  fdic_insured_flag BIT,
  product_status  NVARCHAR(20),
  regulatory_classification NVARCHAR(50),
  source_system   NVARCHAR(50),
  eff_start_date  DATE      NOT NULL,
  eff_end_date    DATE      NOT NULL,
  is_current      BIT      NOT NULL,
  created_ts      DATETIME2,
  updated_ts      DATETIME2,
  CONSTRAINT PK_Bank_Prod_Dim PRIMARY KEY (product_key)
);

```

```

CREATE TABLE silver.Bank_Org_Dim (
  org_key  INT      NOT NULL,
  org_id   INT      NOT NULL,
  org_name NVARCHAR(200),
  org_type_code NVARCHAR(10),
  parent_org_key INT,
  parent_org_id INT,
  status      NVARCHAR(20),
  founded_date DATE,
  country     NVARCHAR(50),
  state       NVARCHAR(10),
  city        NVARCHAR(100),
  address     NVARCHAR(255),
  phone       NVARCHAR(30),
  email       NVARCHAR(100),
  eff_start_date DATE      NOT NULL,
  eff_end_date DATE      NOT NULL,
  is_current   BIT      NOT NULL,
  version_number INT,
  created_ts   DATETIME2,
  updated_ts   DATETIME2,
  CONSTRAINT PK_Bank_Org_Dim PRIMARY KEY (org_key)
);

```

```

CREATE TABLE silver.Customer_Dim (
  customer_key    INT          NOT NULL,
  customer_id     NVARCHAR(20) NOT NULL,
  customer_type   NVARCHAR(20),
  first_name      NVARCHAR(100),
  last_name       NVARCHAR(100),
  business_name   NVARCHAR(200),
  customer_segment NVARCHAR(50),
  credit_score_band NVARCHAR(20),
  address         NVARCHAR(255),
  relationship_status NVARCHAR(20),
  kyc_status      NVARCHAR(20),
  aml_risk_rating NVARCHAR(20),
  primary_org_id  NVARCHAR(20),
  eff_start_date  DATE          NOT NULL,
  eff_end_date    DATE          NOT NULL,
  is_current      BIT           NOT NULL,
  version_number  INT,
  created_ts      DATETIME2,
  updated_ts      DATETIME2,
  CONSTRAINT PK_Customer_Dim PRIMARY KEY (customer_key)
);

CREATE TABLE silver.Fact_Account_GL (
  fact_key    INT          NOT NULL,
  period_date DATE          NOT NULL,
  fiscal_year INT          NOT NULL,
  fiscal_quarter NVARCHAR(5) NOT NULL,
  period_label NVARCHAR(20),
  account_key  INT          NOT NULL,
  account_id   NVARCHAR(20),
  account_status NVARCHAR(20),
  gl_key       INT          NOT NULL,
  gl_code      NVARCHAR(20),
  gl_name      NVARCHAR(255),
  amount_type  NVARCHAR(50),
  customer_key INT          NOT NULL,
  customer_id  NVARCHAR(20),
  product_key  INT          NOT NULL,
  product_id   NVARCHAR(20),
  org_key      INT          NOT NULL,
  org_id       NVARCHAR(20),
  amount       DECIMAL(18,2),
  CONSTRAINT PK_Fact_Account_GL PRIMARY KEY (fact_key),
  CONSTRAINT FK_Fact_GL FOREIGN KEY (gl_key) REFERENCES silver.GL_Hierarchy_Dim
(gl_key),
  CONSTRAINT FK_Fact_Account FOREIGN KEY (account_key) REFERENCES silver.Account_Dim
(account_key),
  CONSTRAINT FK_Fact_Product FOREIGN KEY (product_key) REFERENCES silver.Bank_Prod_Dim
(product_key),
  CONSTRAINT FK_Fact_Org FOREIGN KEY (org_key) REFERENCES silver.Bank_Org_Dim
(org_key),
  CONSTRAINT FK_Fact_Customer FOREIGN KEY (customer_key) REFERENCES silver.Customer_Dim
(customer_key)
);

CREATE INDEX IX_Fact_Period ON silver.Fact_Account_GL (period_date, fiscal_year,
fiscal_quarter);
CREATE INDEX IX_Fact_GL ON silver.Fact_Account_GL (gl_key);
CREATE INDEX IX_Fact_Customer ON silver.Fact_Account_GL (customer_key);
CREATE INDEX IX_Fact_Product ON silver.Fact_Account_GL (product_key);
CREATE INDEX IX_Fact_Org ON silver.Fact_Account_GL (org_key);

```

```
CREATE TABLE silver.B_S_Report_Order (  
  gl_name      NVARCHAR(255),  
  account_category NVARCHAR(50),  
  report_order INT  
)  
;  
CREATE TABLE silver.I_S_Report_Order (  
  gl_name      NVARCHAR(255),  
  account_category NVARCHAR(50),  
  report_order INT  
)  
;
```


A4) Azure SQL DB Silver Layer Stored Procedures

```
CREATE PROCEDURE silver.load_Account_Dim
AS
BEGIN
    SET NOCOUNT ON;

    DELETE FROM silver.Account_Dim;
    INSERT INTO silver.Account_Dim
    SELECT
        account_key, account_id, customer_id, product_id, org_id,
        account_type, account_subtype, subaccount_code, subaccount_name,
        account_status, TRY_CAST(open_date AS DATE),
        interest_rate_type, collateral_type,
        bs_account_code, bs_account_name, bs_account_category, bs_account_subcategory,
        bs_rollup_code, bs_rollup_name, acl_code, acl_name,
        primary_is_code, primary_is_name, primary_is_category,
        secondary_is_code, secondary_is_name,
        TRY_CAST(eff_start_date AS DATE),
        TRY_CAST(eff_end_date AS DATE),
        CASE WHEN is_current = 'True' THEN 1 ELSE 0 END
    FROM bronze.stg_Account_Dim_SCD;
END;
```

```
CREATE PROCEDURE silver.load_GL_Hier_Dim
AS
BEGIN
    SET NOCOUNT ON;

    DELETE FROM silver.GL_Hierarchy_Dim;
    INSERT INTO silver.GL_Hierarchy_Dim
    SELECT
        gl_key, gl_code, gl_name, statement_type,
        account_class, account_category, fs_classification,
        normal_balance, native_sign, parent_gl_code, hierarchy_level,
        CASE WHEN is_leaf = 'True' THEN 1 ELSE 0 END,
        CASE WHEN is_contra = 'True' THEN 1 ELSE 0 END,
        CASE WHEN is_derived = 'True' THEN 1 ELSE 0 END,
        report_order, maps_to_product,
        TRY_CAST(eff_start_date AS DATE),
        TRY_CAST(eff_end_date AS DATE),
        CASE WHEN is_current = 'True' THEN 1 ELSE 0 END
    FROM bronze.stg_GL_Hierarchy_Dim_SCD;
END;
```

```

CREATE PROCEDURE silver.load_Bank_Prod_Dim
AS
BEGIN
    SET NOCOUNT ON;

    DELETE FROM silver.Bank_Prod_Dim;
    INSERT INTO silver.Bank_Prod_Dim
    SELECT
        product_key, product_id, product_name, product_type,
        product_category, product_line, interest_rate, interest_rate_type,
        minimum_balance, monthly_fee, fee_waiver_min_balance, account_term_months,
        product_tier,
        CASE WHEN overdraft_protection_flag = 'Y' THEN 1 ELSE 0 END,
        CASE WHEN digital_banking_flag = 'Y' THEN 1 ELSE 0 END,
        CASE WHEN rewards_program_flag = 'Y' THEN 1 ELSE 0 END,
        CASE WHEN fdic_insured_flag = 'Y' THEN 1 ELSE 0 END,
        product_status, regulatory_classification, source_system,
        TRY_CAST(eff_start_date AS DATE),
        TRY_CAST(eff_end_date AS DATE),
        CASE WHEN is_current = 'Y' THEN 1 ELSE 0 END,
        TRY_CAST(created_ts AS DATETIME2),
        TRY_CAST(updated_ts AS DATETIME2)
    FROM bronze.stg_Bank_Prod_Dim_SCD;

```

```

CREATE PROCEDURE silver.load_Bank_Org_Dim
AS
BEGIN
    SET NOCOUNT ON;

    DELETE FROM silver.Bank_Org_Dim;
    INSERT INTO silver.Bank_Org_Dim
    SELECT
        org_key, org_id, org_name, org_type_code,
        TRY_CAST(parent_org_key AS INT),
        TRY_CAST(parent_org_id AS INT),
        status, TRY_CAST(founded_date AS DATE),
        country, state, city, address, phone, email,
        TRY_CAST(eff_start_date AS DATE),
        CASE
            WHEN TRY_CAST(eff_end_date AS DATE) IS NULL
            THEN '9999-12-31'
            ELSE TRY_CAST(eff_end_date AS DATE)
        END AS eff_end_date,
        CASE WHEN is_current = 'True' THEN 1 ELSE 0 END,
        version_number,
        TRY_CAST(created_ts AS DATETIME2),
        TRY_CAST(updated_ts AS DATETIME2)
    FROM bronze.stg_Bank_Org_Dim_SCD;
END;

```

```

CREATE PROCEDURE silver.load_Customer_Dim
AS
BEGIN
    SET NOCOUNT ON;

    DELETE FROM silver.Customer_Dim;
    INSERT INTO silver.Customer_Dim
    SELECT
        customer_key, customer_id, customer_type,
        first_name, last_name, business_name,
        customer_segment, credit_score_band, address,
        relationship_status, kyc_status, aml_risk_rating, primary_org_id,
        TRY_CAST(eff_start_date AS DATE),
        TRY_CAST(eff_end_date AS DATE),
        CASE WHEN is_current = 'True' THEN 1 ELSE 0 END,
        version_number,
        TRY_CAST(created_ts AS DATETIME2),
        TRY_CAST(updated_ts AS DATETIME2)
    FROM bronze.stg_Customer_Dim_SCD;
END;

```

```

CREATE PROCEDURE silver.load_Fact_Account_GL
AS
BEGIN
    SET NOCOUNT ON;

    DELETE FROM silver.Fact_Account_GL;
    INSERT INTO silver.Fact_Account_GL
    SELECT
        fact_key, TRY_CAST(period_date AS DATE),
        TRY_CAST(fiscal_year AS INT),
        fiscal_quarter, period_label,
        account_key, account_id, account_status,
        gl_key, gl_code, gl_name, amount_type,
        customer_key, customer_id,
        product_key, product_id,
        org_key, org_id, amount
    FROM bronze.stg_Fact_Account_GL;
END;

```

```

CREATE VIEW gold.vw_B_S_Report_Order AS
SELECT
    f.gl_name,
    f.account_category,
    f.report_order
from silver.B_S_Report_Order f;

```

```

CREATE VIEW gold.vw_I_S_Report_Order AS
SELECT
    f.gl_name,
    f.account_category,
    f.report_order
from silver.I_S_Report_Order f;

```

5) Azure SQL DB Gold Layer Views

```
CREATE VIEW gold.vw_Income_Statement AS
SELECT
  f.period_date,
  f.fiscal_year,
  f.fiscal_quarter,
  f.period_label,
  gl.account_class,
  gl.account_category,
  gl.fs_classification,
  gl.gl_code,
  gl.gl_name,
  gl.report_order,
  gl.normal_balance,
  gl.native_sign,
  gl.is_contra,
  f.amount_type,
  SUM(f.amount * gl.native_sign) AS signed_amount,
  SUM(f.amount) AS raw_amount,
  c.customer_segment,
  c.credit_score_band,
  p.product_type,
  p.product_category,
  p.product_line,
  o.org_name,
  o.org_type_code,
  o.state,
  o.city,
  rpt.placement,
  rpt.display_order AS section_order
FROM silver.Fact_Account_GL f
JOIN silver.GL_Hierarchy_Dim gl ON f.gl_key = gl.gl_key AND gl.is_current
= 1
JOIN silver.Customer_Dim c ON f.customer_key = c.customer_key AND c.is_current
= 1
JOIN silver.Bank_Prod_Dim p ON f.product_key = p.product_key AND p.is_current
= 1
JOIN silver.Bank_Org_Dim o ON f.org_key = o.org_key AND o.is_current
= 1
LEFT JOIN silver.BS_IS_Report_Order rpt ON gl.account_class = rpt.account_class
AND rpt.statement = 'IS'
WHERE gl.statement_type = 'IS'
AND gl.is_leaf = 1
GROUP BY
  f.period_date, f.fiscal_year, f.fiscal_quarter, f.period_label,
  gl.account_class, gl.account_category, gl.fs_classification,
  gl.gl_code, gl.gl_name, gl.report_order, gl.normal_balance,
  gl.native_sign, gl.is_contra, f.amount_type,
  c.customer_segment, c.credit_score_band,
  p.product_type, p.product_category, p.product_line,
  o.org_name, o.org_type_code, o.state, o.city,
  rpt.placement, rpt.display_order;
```

```

CREATE VIEW gold.vw_Balance_Sheet AS
SELECT
  f.period_date,
  f.fiscal_year,
  f.fiscal_quarter,
  f.period_label,
  gl.account_class,
  gl.account_category,
  gl.fs_classification,
  gl.gl_code,
  gl.gl_name,
  gl.report_order,
  gl.normal_balance,
  gl.native_sign,
  f.amount_type,
  SUM(f.amount * gl.native_sign) AS signed_amount,
  SUM(f.amount) AS raw_amount,
  c.customer_segment,
  p.product_type,
  p.product_category,
  o.org_name,
  o.org_type_code,
  o.state,
  rpt.placement,
  rpt.display_order AS section_order
FROM silver.Fact_Account_GL f
JOIN silver.GL_Hierarchy_Dim gl ON f.gl_key = gl.gl_key AND gl.is_current
= 1
JOIN silver.Customer_Dim c ON f.customer_key = c.customer_key AND c.is_current
= 1
JOIN silver.Bank_Prod_Dim p ON f.product_key = p.product_key AND p.is_current
= 1
JOIN silver.Bank_Org_Dim o ON f.org_key = o.org_key AND o.is_current
= 1
LEFT JOIN silver.BS_IS_Report_Order rpt ON gl.account_class = rpt.account_class
AND rpt.statement = 'BS'
WHERE gl.statement_type = 'BS'
AND gl.is_leaf = 1
GROUP BY
  f.period_date, f.fiscal_year, f.fiscal_quarter, f.period_label,
  gl.account_class, gl.account_category, gl.fs_classification,
  gl.gl_code, gl.gl_name, gl.report_order, gl.normal_balance,
  gl.native_sign, f.amount_type,
  c.customer_segment, p.product_type, p.product_category,
  o.org_name, o.org_type_code, o.state,
  rpt.placement, rpt.display_order;

```

```

CREATE VIEW gold.vw_I_S_Report_Order AS
SELECT
  f.gl_name,
  f.account_category,
  f.report_order
from silver.I_S_Report_Order f;

```

```
CREATE VIEW gold.vw_B_S_Report_Order AS
SELECT
    f.gl_name,
    f.account_category,
    f.report_order
from silver.B_S_Report_Order f;
```

Power BI & Azure SQL: Financial Reporting Exercise

Author: Al Franco | July 2026 | Portfolio Project